

Algoritmos y Estructuras de Datos

Cadenas de caracteres



Contenidos del documento:

- ❖ Caracteres vs strings en C# y conceptos de Inmutabilidad.
- ❖ Recorrido secuencial de cadenas.
- ❖ Operaciones típicas: longitud, Interpolación, extracción, concatenación, Subcadenas, Dividir y unir.
- ❖ Comparación y ordenamiento alfabético.
- ❖ Búsqueda y reemplazo

Caracteres vs Strings:

Aunque parezcan similares, su naturaleza técnica en C# es distinta:

char: Representa un único carácter Unicode (16 bits). Se define con comillas simples (ej. 'A'). Es un tipo de valor.

string: Es una secuencia de objetos char. Se define con comillas dobles (ej. "Hola"). Es un tipo de referencia.

El Concepto de Inmutabilidad

Esta es la característica más importante: los strings en C# son inmutables.

Una vez que creas un objeto string, no puedes cambiar su contenido. Si intentas "modificarlo" (por ejemplo, pasarlo a mayúsculas), .NET en realidad crea un nuevo objeto en memoria con el cambio y desecha el anterior.

Recorrido secuencial de cadenas:

Un string se puede tratar como una colección de caracteres de solo lectura. Podes acceder a cada posición mediante un índice, empezando desde 0.

Bucle for: Útil si necesitas conocer la posición (índice).

```
string texto = "Programo en c#";  
  
for (int i = 0; i < texto.Length; i++) {  
    char letraActual = texto[i];  
    Console.WriteLine(letraActual);  
}
```

Bucle foreach: La forma más limpia si solo necesitas el carácter.

```
string texto = "C#";  
  
foreach (char c in texto)  
{  
  
    Console.WriteLine(c);  
  
}
```

Operaciones Típicas:

Operación	Método/Sintaxis	Descripción
Longitud	.Length	Propiedad que devuelve el número de caracteres.
Interpolación	(\$"{var}")	La forma moderna y legible de insertar variables en texto.
Extracción	texto[0]	Acceso directo a un carácter por su índice.

Concatenación	<code>+ o string.Concat()</code>	Une dos o más cadenas.
Subcadenas	<code>.Substring(inicio, largo)</code>	Extrae una porción de la cadena original.
Dividir	<code>.Split(',')</code>	Corta un string en un array basado en un separador.
Unir	<code>string.Join('-', array)</code>	Toma un array y lo une en un string con un separador.

Operaciones Típicas:

Comparar strings no es solo ver si son iguales, sino determinar su orden en un diccionario.

- **Igualdad** (`==` o `.Equals`): Compara si el contenido es idéntico.
- **Ordenamiento** (`.CompareTo`): * Devuelve 0 si son iguales.
 - Devuelve < 0 si la primera va antes alfabéticamente.
 - Devuelve > 0 si la primera va después.

Busqueda y Reemplazo:

Búsqueda

- **.Contains("abc")**: Devuelve un booleano si encuentra la subcadena.
- **.IndexOf('c')**: Devuelve la posición de la primera aparición (o -1 si no existe).
- **.StartsWith() / .EndsWith()**: Verifica el inicio o fin del texto.

Reemplazo

- **.Replace("viejo", "nuevo")**: Genera una nueva cadena donde todas las ocurrencias del primer parámetro son sustituidas por el segundo.
- **Búsqueda/Reemplazo Manual**: Si no existieran estos métodos, tendrías que recorrer el string con un bucle, identificar el índice de coincidencia y reconstruir la cadena usando `Substring` y concatenación.

